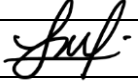
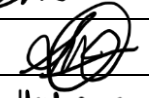
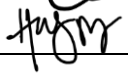


TFB1103 SOFTWARE ENGINEERING

Hotel Management System

No	Student Name	Student ID	Task Completed	Mark
1.	Nur Fathiah Aisha Bt Ramniza	22011102	Section 1 & Section 3	
2.	Nur Afina Madiah Bt Muhammad Mazrin	22011697	Section 2	
3.	Nur Shalili Amira binti Badrul Hisham	22011687	Section 1 & Section 2	
4.	Nur Husna Syaza binti Zainal Abidin	22011693	Section 3	

Task	Marks	Comment
Section 1: Introduction	/5	
Project background	/5	
Objectives		
Section 2: System Requirements	/10	
20 functional requirements,	/10	
10 non-functional requirements,	/10	
10 constraints,	/5	
block diagram/use-case diagram		
Section 3: System Architecture	/10	
Architecture	/10	
Explanation	/15	
Justification		
Total	/80	

Table of Contents

Section 1: Project Introduction.....	3
1.1 Project Background	3
1.2 Objectives	4
Section 2: System Requirements.....	6
2.1 20 Functional Requirements	6
2.2 10 Non-Functional Requirements.....	10
2.3 10 Constraints.	12
2.4 Use Case Diagram	14
Section 3: System Architectural Pattern	15
3.1 Draw an Architectural Pattern for Your Proposed Business	15
3.2 Explain Proposed Architectural Pattern	19
3.3 Justification of the Proposed Architectural Pattern	20

Section 1: Project Introduction

1.1 Project Background

During recent years, the hospitality industry has undergone various transformations associated with the evolution of customer expectations and the rapid rate of technological innovation. However, most hotels still rely on manual practices or uncoordinated systems to execute the most crucial operations such as room bookings, customer complaints, and housekeeping management. These fragmented operations are likely to cause low efficiency, human errors and overbooking, and slow services, which ultimately cause poor performance of operations and customer satisfaction.

In response to such obstacles, this project suggests that the creation of an integrated hotel management system that will simplify the internal processes within a hotel and enhance the experience enjoyed by guests should be developed. The system limits the use of manual input in the process of scheduling check-ins, check-outs, assigning rooms, and schedules to cover housekeeping in the hotel, thus reducing human error. This automation plays a significant role in increasing the productivity of the staff, increasing accuracy, and promoting prompt service delivery by departments.

Meanwhile, one must consider the end-user, which means this is the program that will be used by the staff and guests at the hotel. The interface should be functional, smooth, by means of which visitors will be able to read about the availability of rooms in minutes, make safe bookings and payments, and leave personal feedback without any difficulties. These qualities can assist in making interaction with the hotel and, ultimately, developing loyalty and positive reviews better and smoother.

In addition, the project involves a strong element of data analytics that would assist hotel management in making business decisions. The system empowers managers through providing insights since it automatically reports on key performance measures such as occupancy rates, revenue streams, and customer satisfaction among others. Such a data-driven strategy helps to increase planning, pricing decisions and resource management and eventually leading to long-term success of a hotel in a competitive market.

This project combines the concept of operational effectiveness, design customer orientation and management smarts in a way that expresses contemporary, end-to-end solutions to the old problems of managing hotels.

1.2 Objectives

1. To streamline hotel procedures by automating some of the main tasks such as booking rooms, receiving or releasing guests as well as organizing the housekeeping services.

Hotel maintenance is often carried out with manual records or systems that do not integrate, necessitating delays, overbooking, and mismanagement among the staff members. This objective aims to facilitate the automation of daily tasks and reunite them under a single platform. This will help speed up the process of check-ins and check-outs, ensure that the rooms are allocated appropriately, and help to maintain cleaning in real time. This facilitates the work of hotel staff, and statistically reduces the possibility of error.

2. To enhance the experience of a guest, offering them an easy and intuitive system to engage with, that displays room availability in real time, allows quick bookings, takes secure payments, and gathers personalized feedback.

The success of a hotel is determined by the degree of satisfaction enjoyed by its clients. This intention aims at putting in place a system which would make the use of the hotel services by the guests very easy, either when booking a room, checking-in, or making payment. Giving the guest real-time information and a clear, secure interface, the system eases and secures them in the process of experiencing the same. Post-stay feedback tools can also be used in the hotel for improvements in the provision of various services, depending on what actual clients state.

3. To provide the management with helpful information in terms of automated reports on occupancy rates, revenue, and customer satisfaction and thus assist them to make informed decisions based on the data leading to the success of the firm.

Hotel management requires timely and accurate data to make a wise commercial decision. This objective concerns the analytical component of the system, where automatic reports and visible dashboards enable the management to monitor such crucial performance indicators. Revenue,

occupancy trends, and guest feedback data in real-time allow decision-makers to understand how to make better improvements, to set prices more efficiently, and to plan further growth or resources allocation.

Section 2: System Requirements

2.1 20 Functional Requirements

1. User & Role Management

- i) The system shall allow Admins, Receptionists, and Guests to register/login with role-based access

This will make sure that every type of user will be able to access just the functionality that they are subjected to. Admins can use all the administrative options, the Receptionists will manage booking and check-ins, and the Guests will explore personal booking capabilities. Role-based access is more secure, and chances of unauthorized activities are minimized.

- ii) The system shall allow Admins to create and manage user profiles.

Employees will be able to whitelist newly enrolled users, refresh the information on the users, and maintain accounts. The functionality assists in ensuring and keeping records of users that are precise and current. It allows quick processing of guest requests and customization of services.

- iii) The system shall allow Admins to **reset passwords and deactivate users.**

Admins may help users who lost their passwords, by allowing them to reset their details in a safe manner. They are also capable of disabling user accounts when there is a policy violation or mismanagement of an account. This increases control over the system and eliminates skullduggery.

- iv) The system shall allow guests to update their own profile and preferences.

The guests are allowed to log-in and to update their personal information like contact details or booking rooms of type. This enhances a more personalized version as well as minimizing staff's reliance on trivial adjustments. It also keeps the database up to date.

2. Room Booking and Reservation

- i) The system shall allow guests to view available rooms by type, date, and price.

Customers will be capable of filtering and searching rooms according to their demands. This option assists them to locate rooms that suit them based on their needs and affordability. Real time data guarantees that rooms that are presented on the screen are in real-time condition.

- ii) The system shall allow guests to make new reservations and receive confirmation. Booking is done via the system interface, allowing guests to choose a room. After this confirmation has been done, the system will issue confirmation of an instant booking and every detail therein. This gives orientation and confidence to the visitor.

- iii) The system shall allow guests to cancel or modify bookings.

The guests will be allowed to change the reservation dates or the room type or even cancel the booking in case they have a change of plans. This saves on manpower of employees and gives flexibility to customers. These changes will be updated immediately at the system.

- iv) The system shall automatically update room availability in real time

Once there is a reservation, modification, or cancellation, the system will update the room status. This will avoid overbooking and real-time monitoring of rooms. It also makes sure that all users are aware of their present availability.

3. Check-In and Check-Out

- i) The system shall allow receptionists to check in on guests upon arrival and assign rooms.

The system allows receptionists to check or confirm the required details of the guest and assign the guest to the correct room. Check-in will take a short time, and it will be organized. It will also change the status of the occupied room.

- ii) The system shall allow receptionists to process check-out and calculate billing.

When the guest checks out, the receptionist can settle the bill according to length of stay and any extra expenses. Totals will be automatically calculated by the system. This keeps the correct billing, and the process of checking out is faster.

- iii) The system shall allow automated invoice generation and printing.

Once the payment has taken place, the system will create a complete invoice that can be either printed or emailed. This bill contains the hotel room fees, taxes, as well as other additional services utilized. The automation decreases the errors and offers an opportunity of having a professional checkout.

4. Payment and Billing

- i) The system shall support multiple payment methods (credit card, cash, e-wallet).

Different payment methods will be available to guests on both checking out and checking in. This provides flexibility, which increases convenience and addresses the preferences of customers. It also facilitates easier processing of transactions.

- ii) The system shall send email receipts to guests after payment.

After the payment has been made, the system will automatically create a digital receipt and send a copy to the guest via email. This gives evidence of transactions and enhances customer satisfaction. It additionally helps to minimize receipt printing costs manually.

- iii) The system shall track outstanding balances or deposits for each guest.

The system will keep track of the unpaid balances, and any security check-in deposits collected and easily accessible and visible by staff and directory and can be viewed and managed as a check-out billing or direct refund. This makes financial accuracy and transparency.

5. Housekeeping and Maintenance

- i) The system shall allow staff to assign cleaning tasks to housekeeping.

Receptionists may also allocate rooms requiring cleaning depending on check-outs by the guests or daily plan of work. The system allows tracking tasks to make sure that they are completed on time. This assists in the standards of cleanliness.

- ii) The system shall allow maintenance staff to log and resolve room issues.

Maintenance personnel are able to file complaints like damaged fixtures or defective machinery in the system. Every task will contain such information as the room number, the description of the problem, and whether it is solved. This makes the maintenance processes flow smoothly.

- iii) The system shall notify receptionists when tasks are delayed or unresolved.

Receptionists will get system warnings in case tasks are yet to be accomplished within a predetermined time frame. This will enable them to follow up, reassignment, or escalation of the problem. It provides efficiency in its operations, and it secures guest satisfaction.

6. Reporting and Feedback

- i) The system shall generate occupancy, revenue, and guest reports for admin.

Admin users can create automatic reports summarizing data of possible interest of a hotel such as bookings, revenue trends and guest demographics. Such reports are helpful in strategic planning and day-to-day management of choices. Reports can be viewed or exported.

- ii) The system shall allow guests to submit ratings and feedback after their stay.

Upon check-out, the guests will then give their views regarding the experience. They are able to rank hygiene, customer service, and general satisfaction. This information will assist the hotel in determining areas of strength and weakness.

- iii) The system shall allow admin to review and export feedback reports.

All the guest feedback is available to admins and can be categorized in one place, and reports can be generated. These reports make it possible to supervise quality in the services as time goes by. They are exportable with the purpose of sharing or analysis by other members of an organization

2.2 10 Non-Functional Requirements

1. Performance

The system shall be capable of handling at least 100 concurrent users including hotel staff and guests without any significant delay. It will perform common tasks such as booking, check-ins, check-outs, and report generation with maximum response time. This ensures that the system remains efficient, even during peak periods or high occupancy seasons.

2. Security

The system will create a secure environment to protect sensitive, personal guest information as well as payment information. This includes the use of end-to-end encryption protocols like HTTPS with TLS 1.3 to secure data in transit, as well as secure authentication mechanisms. Admin accounts will require a multi-factor authentication combination and strong passwords so that guests' data can only be accessed by authorized personnel to critical features.

3. Reliability

To provide uninterrupted service, the system will be committed to reliability in ensuring the system will have uptime to achieve maximum reliability, which limits downtime to an annual maximum of 8.76 hours. Also, automated backup is performed daily that protects hotel data and enables restoration of hotel related past data, making it quicker to recover from unwanted failures and minimize hotel disruption.

4. Usability

The system will have an intuitive and easy-to-use interface for both the staff and guests, leveraging an environment in which the staff and guests can quickly and conveniently facilitate core activities like booking rooms, checking in and checking out and generating reports. The system will be designed in such a way that a new user can navigate the system and perform the core functions within five minutes of using it for the first time, and without resorting to technical support. This will improve operational efficiency and customer satisfaction.

5. Maintainability

The system will have increased ease with modular architecture as it will allow individual components to be updated or fixed without interfering with the system. This structure will allow developers to deliver system enhancements or fix serious bugs in 48 hours, while allowing easier upgrades and scaling for long-term use cases. Maintainability means the hotel management system will remain adaptable and sustainable as operational needs change.

6. Portability

The system will be easily transportable to different hardware and software setups without the need for significant reconfiguration. It will offer an automatic installation procedure that makes installation easier when it comes to setting up or for a remote user or branch of a hotel. Furthermore, it will be implementable on different operating systems, including Windows, macOS, and Linux, which allows for long-remaining flexibility and ease of deployment.

7. Fault Tolerance

The system will be fault-tolerant, whereby the system will remain operational in case of failure by a component such as booking module or payment service. Error handling processes and built-in fail mechanisms will help contain faults without total shutdown of a system. This guarantees uninterrupted services and reduces interference with the user.

8. Changeability

The system will have new changes in business rules such as price variation and booking conditions, so it could be implemented with minor changes to the codebase. Whenever possible, configuration files and admin panels will be applied so that the developer's intervention is minimal. This makes the hotel flexible enough to change its operation or policy without delays.

9. Install ability

The system will possess user-friendly and guided installation, which enables installation in 10 minutes. Minor technical skills will be required to install it, and automated checks will be provided by the environment. This will help to minimize set-up mistakes and provide quicker running time for non-technical personnel.

10. Understandability

The structure and workflow of the system will be introduced in a manner that users would find it easy to understand the overall intention and rationality of all the features of the system. Clearly labeled, standardized terminologies and meaningful labels will assist the user to know how the various components of the system relate and interact. This will lessen the dependency of the users on training manuals, and we will be able to make a faster mental mapping of how to use the system.

2.3 10 Constraints.

1. Development process

The system is expected to be developed utilizing an Agile process, and working with hotel staff during each sprint will ensure real world usability across all areas such as front desk, housekeeping, and management.

2. Budget

The total cost of development, deploying the system, and training hotel staff must not exceed RM200,000 for the consultative project period of the first 12 months.

3. Deadlines

The timeline for the project deliverables is that the system will be delivered for your operational use within 6 months, while a fully functional prototype system will be delivered for your evaluation within the first 3 months.

4. Team

The project team will be limited to 6 individuals, with at least one hospitality industry consultant and one representative from the hotel staff.

5. Legislation

The system must assure compliance with Malaysia's PDPA for guest data protection, and any relevant legal requirements for digital financial transactions and guest records.

6. Norms and Culture

The norms and culture of the local area must be reflected in the system's interface and workflows including factors such as language options, polite tone in communication and local date time formats.

7. Standards

The system must conform to HTNG (Hotel Technology Next Generation) standards for API integration with third-party booking platforms, POS systems and secure payment gateways.

8. Operations

In executing the implementation, the hotel must use either a phased rollout or parallel operating strategy to ensure all operational capacities remain available during implementation specifically during check-in or check-out and billing.

9. Guidelines

As the aim is to have an easy-to-use system available to all hotel personnel regardless of their ability to operate technology, the design and user interface must adhere to usability principles such as Nielsen's usability heuristics.

10. Security

Role-based permissions and multi-factor authentication (MFA) must be implemented for all staff access, while in-transit and at-rest encryption must be employed for guest and payment data.

2.4 Use Case Diagram

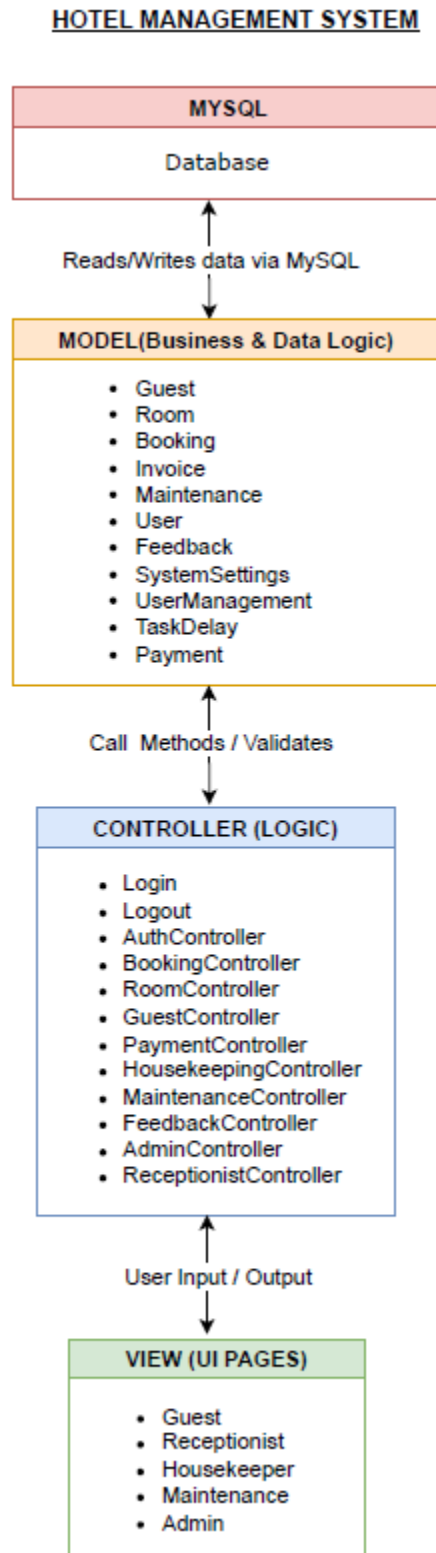




Section 3: System Architectural Pattern

3.1 Draw an Architectural Pattern for Your Proposed Business

Model-View-Controller (MVC) architectural pattern is selected for the development of the Hotel Management System because it clearly separates concerns, has a modular structure, and work well for web-based applications. This design breaks the program up into three parts that work together which is the Model (data and business logic), the View (user interface), and the Controller (request processing and coordination). This makes it possible to develop the app in parallel, test it more easily, and keep it up to date. MVC is a great choice for hotel systems since different types of users, use different interfaces to interact with the system. It also helps maintain business logic consistent, secure, and scalable. Because of its layered design, it can be easily upgraded in the future with microservices integration or links to third-party APIs.

Model-View-Controller (MVC) architectural pattern

DATABASE

Tables

- Guests
- Rooms
- Bookings
- Payments
- Feedback
- Users
- Admin
- Receptionist
- HouseKeeper
- Maintenance
- Housekeeping_task
- Maintenance_requests
- Invoices

MODEL (Business and Data Logic)

- Guest: Handles guest registration, profile management, reservation history, and payment records
- Room: Manages room availability, type, assignments, housekeeping status
- Booking: Stores and track reservation data, booking status, check in/out
- Invoice: Generates and stores invoices linked to bookings and payments; maintains invoice history
- Maintenance: Manages maintenance tasks: issue logging, technician assignment, status updates
- User: Stores login credentials, admin, guest, receptionist, housekeeper, maintenance, and activity logs
- Feedback: submission, admin review
- SystemSettings: configurable system parameters
- UserManagement: user CRUD, assign roles
- TaskDelay: Tracks reasons and timestamps of task delays; links to tasks affected
- Payment: Handles transaction processing, payment methods, and status tracking

CONTROLLER

Controller	Responsibilities
Login	Validate credentials, Set session/token, Redirect to role-specific dashboard
Logout	Clear session/token, Redirect to login page
AuthController	Login, registration for guests, staff, and admin
BookingController	Make, modify, cancel reservations; check-in/out, booking confirmation
RoomController	Assign rooms, fetch room availability
GuestController	Profile updates, invoice viewing
PaymentController	Process payments, check status and generate invoices
HousekeepingController	View assigned tasks, update task progress
MaintenanceController	Log issues, update task progress and view assigned task
FeedbackController	CreateFeedback(), ViewAllFeedback(), FilterByUser()
AdminController	Manage users, configure system, view & export report, perform create/ update/ delete users
ReceptionistController	Coordinate guest registration, assign room, initiate check in/out, generate invoices, assign task to housekeeping and maintenance, check task status, receive delay notification, view guest profile

VIEW (UI PAGES)

User Role	UI Pages (View)
Guest	Register, view rooms, make reservation, make payment, view invoice, submit feedback, Login Page, Logout, notification page
Receptionist	Guest profiles, check-in/out, assign rooms, log maintenance, assign housekeeping, check task status, notification page, Login Page, Logout
Housekeeper	View tasks, update task status, notification page, Login Page, Logout
Maintenance	View assigned issues, log new issue, update status, Login Page, Logout, notification page
Admin	Manage users, configure settings, review feedback, view & export reports, Login Page, Logout

3.2 Explain Proposed Architectural Pattern

The Model View Controller pattern organizes the system into three distinct layers, each with a unique function, while working together to deliver full functionality.

The Model layer acts as the foundation of the system, containing major components that handle business logic and data operations. Important modules include the Guest profile manager, which takes care of registrations and keeps personal information, the Room inventory system, which monitors room availability and status, the Booking engine handles everything related to reservation management and modifications, and the Payment processor, ensuring secure financial transactions. Every one of these modules guarantees data precision, enforces company policies, and aids vital hotel functions.

The View layer signifies the presentation tier of the system, crafted primarily with user experience in mind. It comprises carefully crafted interfaces tailored for each user role in the hotel ecosystem. Visitors utilize reservation forms and account management interfaces, while receptionists operate check-in and room allocation terminals, and administrators oversee system performance through dashboards. The View layer emphasizes transparency, brand uniformity, and compatibility across platforms. Adaptable layouts and accessibility options have been placed to meet varied user requirements.

By overseeing data interchange and user interactions, the Controller layer serves as a link between the Model and View. Every major system function has its own controller. For instance, all reservation-related operations like creating new bookings and modifying or canceling existing ones, are managed by the BookingController. The PaymentController handles financial transactions, making sure they are secure, while the AdminController oversees user management and system configurations. These controllers verify inputs, engage with the Model, and decide the suitable response, maintaining uniform behavior and a distinct separation between logic and presentation.

There are definite benefits to the MVC architecture in terms of long-term system maintenance and development efficiency. Different teams can work on components concurrently without creating dependencies or conflicts because of the layers' obvious separation. For instance,

the user interface design team can refine the View components while the database team optimizes the Model layer, with both groups coordinating through well defined controller interfaces. This modular approach also simplifies testing procedures as each layer can be verified independently before integration. Another big benefit of MVC is that it makes unit and integration testing easier. Each layer, including Model, View, and Controller, may be tested independently. You can test business logic without loading UI components, and you can check how responsive the UI is using fake data. This separation accelerates debugging and lowers the likelihood of regressions, which improves the software.

Furthermore, architecture inherently facilitates iterative development processes, rendering it perfect for our Agile-driven project schedule where functionalities can be developed, tested, and launched progressively.

3.3 Justification of the Proposed Architectural Pattern

The MVC design is the greatest solution for our Hotel Management System when we look at the most significant system needs from a few different aspects.

The MVC architecture is safer since it breaks up important activities into smaller components. The Model layer keeps all the business logic and data access in one location, which makes it easier to put security measures in place. At the Controller level, role-based access control handles both user authentication and authorisation. This makes sure that guests, staff, and administrators can only utilise features that are appropriate for their level of access. There are a lot of different levels of data validation. For instance, the Controller cleans up inputs before sending them to the Model, and database queries are set up with parameters to stop injection attempts. Also, the separation of concerns makes it less likely that security weaknesses would spread through the complete system because breaches are limited to one layer.

Performance was a significant reason we chose this architecture because the hotel needs to constantly be able to serve customers quickly. You can apply optimisation tactics at each level of the MVC framework to make sure everything works well. The Model layer can handle a lot of reservation requests during busy times thanks to advanced data caching and query optimisation.

The Controller layer is in charge of directing requests and balancing the load so that system resources are used well. The View layer uses client-side rendering to make it seem like the system is running faster for end users and to take some of the load off the server. This strategy of performance optimisation with multiple layers allows us meet our rigorous reaction time requirements while keeping the system stable even when the load changes. The architecture also lets you do things like make reports in the background, which means that these tasks won't slow down the main booking operations.

We also thought a lot about how scalable our architectural choice would be. The MVC design makes it easy for a business to grow by using both vertical and horizontal scaling methods. You may change the size of each part separately based on how much people want it. For example, at busy times of the year, you can use additional copies of the booking controller, and during the holidays, you can make the components that process payments bigger. When the presentation, business logic, and data access are clearly separated, it is easy to share parts between servers or cloud services.

The MVC architecture is modular, thus it can operate with other systems as well. Hotels really require this because they may need to connect to payment gateways, outside booking platforms like Agoda and Booking.com, and government tourism databases. The system can change to work with new protocols or platforms without changing how it operates or how users interact with it by putting these connections in separate controller modules. The system architecture may grow with the hotel as it gets bigger or opens more locations, so it doesn't need to be fully redone. MVC's modular design also makes it easier to use well-defined APIs to integrate other services, such incentive programs or third-party booking sites.

The MVC technique meets all of the users' needs for help. Separating View parts lets you design bespoke interfaces for users with different responsibilities and levels of technical skill. Guests may quickly choose and reserve rooms thanks to simple, easy-to-use interfaces. On the other hand, staff members can use successful operational solutions that are made just for how they work. The design lets you show information in a lot of different ways, from mobile-friendly displays for visitors to data-rich administrative interfaces. It does this by employing shared Controller logic to make sure everything stays the same. This design philosophy puts the user first and includes features that make the system easier to use for persons with impairments, support for

several languages, and interfaces that can be altered without affecting how the system operates. The end result is a system that makes all user groups more productive at their tasks while reducing the requirement for training.

We also considered a lot about how easy it would be to keep up and how well it would work in the future while we were picking an architecture. It's considerably easier to preserve and upgrade the system over time because the MVC design makes it apparent what each part does. You may add new features and solve faults without affecting other areas of the system too much. For example, if you wanted to add more ways to pay to the payment processing module, you would mostly need to change the Payment Model and Controller. You wouldn't have to update any of the Views that guests see unless you wanted new parts for the interface. Because of this modularity, regression testing is less essential and the risk of system modifications is lower. It is very straightforward to use continuous integration and deployment with this architecture. This helps you make minor, regular changes to the system with very little downtime.

The MVC design meets all of our nonfunctional needs, like reliability, availability, and fault tolerance. You can deploy important elements more than once, and the Controller level controls how to handle failovers. The gap between the layers acts as natural barriers that minimise the effects of component failures to a minimal. For instance, if the reporting module isn't working, the primary functionality for making reservations and checking in will still work. The architecture also lets you log and monitor everything at every level, which makes it easy to detect and fix problems with the system before they happen. At the Controller level, you can make the design more fault-tolerant by using retry patterns, circuit breakers, and backup services. For instance, if the payment gateway is down for a short time, a fail-safe controller can put the request in a queue and try again later. This makes sure that everything runs more smoothly and that the user has the best experience possible. These capabilities are needed to keep the system running 24/7 at a hotel, because system downtime directly affects guest happiness and the efficiency of the business.

To sum up, the Model View Controller architectural pattern is the ideal solution for our Hotel Management System since it fits all of our objectives for security, performance, scalability, user experience, and ease of maintenance. It has worked well in previous business situations and fits with our project's specific aims, therefore it is the ideal choice for giving us a powerful, efficient, and future-proof solution. The architecture not only meets our needs right now, but it also

lets us make modifications and grow the business in the future, which will make the hotel functioning useful for a long time. The architectural diagram that accompanies with this shows how these pieces work together to build a system that is both cohesive and high performing, so that it works well for both guests and staff.